Manual de Construção no Minecraft para uma IA Local

Objetivo: ensinar um agente de IA a planejar, construir, validar, corrigir e aprender construções Minecraft sem reduzir objetos complexos a caixas ou cubos.

Este documento foi escrito para ser ingerido por uma KnowledgeBase/RAG. As regras são deliberadamente explícitas, repetíveis e testáveis.

PRINCÍPIO CENTRAL

Construir não significa apenas colocar blocos. Construir significa transformar um conceito em componentes, relações espaciais, proporções, materiais, circulação e silhueta reconhecível. A execução só deve começar depois que o plano puder ser validado.

```
PEDIDO -> CLASSIFICAR -> DECOMPOR -> DIMENSIONAR -> PLANEJAR
-> VALIDAR -> EXECUTAR -> OBSERVAR -> CORRIGIR -> VERIFICAR -> RESPONDER
```

1. Regras universais

• Antes de qualquer construção não trivial, identifique a categoria do objeto, a escala, o mundo, a origem, os componentes essenciais e o tipo de geometria dominante.

• Não use CUBOID ou HOLLOW_CUBOID como fallback universal. Cubos são componentes, não substitutos para forma.

• Uma construção sem componentes essenciais deve falhar na validação antes de tocar no mundo.

• Prefira operações em lote e primitivas geométricas. Evite milhares de set_block individuais.

• Não construa dentro do jogador. Calcule bounds e uma origem segura.

• Para estruturas grandes, execute por fila com budget de blocos por tick.

• Registre todos os blocos alterados para permitir undo exato.

• Depois de construir, compare o resultado com o pedido original. Se houver divergência clara, corrija antes de dizer 'pronto'.

2. Planejamento espacial

• Use um sistema local de coordenadas. A origem da estrutura deve ser (0,0,0) no plano; só depois transforme para coordenadas do mundo.

• Escolha um eixo principal. Aviões e navios normalmente têm um eixo de comprimento; casas têm eixos de fachada/profundidade; pontes têm eixo entre endpoints.

• Defina bounds antes de executar. Todo componente precisa ficar dentro desses bounds ou ampliar o plano conscientemente.

• Use proporções relativas, não apenas números fixos. Ex.: largura = comprimento * 0.14.

• Use simetria apenas quando o objeto pede simetria. Formas naturais devem ter ruído e assimetria controlada.

• Evite paredes longas perfeitamente lisas quando a categoria pede organicidade ou detalhe.

3. Primitivas geométricas recomendadas

Primitiva	Uso correto	Evitar
CUBOID	cômodos, fundações, volumes internos	avião inteiro, ilha inteira, árvore inteira
HOLLOW_CUBOID	salas simples, caixas explicitamente pedidas	arena genérica, casa final, bandeira
LINE / CURVED_LINE	cabos, vigas, caminhos, fuselagens-guia	paredes grossas inteiras
ELLIPSE / ELLIPSOID	corpos orgânicos, cavernas, copas, fuselagens	objetos que precisam cantos retos
TAPERED_PRISM	casco, fuselagem, torres afuniladas	salas retangulares comuns
EXTRUDED_POLYGON	asas, telhados, cascos, símbolos	uso indiscriminado
NOISE_CARVE	cavernas e terreno natural	arquitetura

4. Casas e edifícios

Uma casa não é uma caixa com porta. Planeje: fundação, fachada, entrada, janelas, telhado, divisões internas, circulação, piso, iluminação e mobiliário básico.

Para casa pequena, uma largura de 7-13 blocos e profundidade de 7-15 costuma ser coerente. O telhado deve ultrapassar a parede 1-2 blocos e ter inclinação.

Evite paredes externas totalmente lisas: use pilares, molduras, variação de profundidade, beirais ou materiais secundários.

Em edifícios de apartamentos, repita módulos de unidades, mas inclua corredor/escada e fachada coerente.

5. Aviões

O avião deve ser reconhecível pela silhueta antes dos detalhes. Componentes essenciais: fuselagem alongada, nariz afunilado, cockpit, duas asas, cauda vertical, estabilizadores horizontais; motores quando apropriado.

Regra aproximada: largura da fuselagem = 10-18% do comprimento; envergadura = 75-115% do comprimento; asa principal no meio do corpo; cauda menor e traseira.

Não faça fuselagem como um prisma retangular constante. Use seções que crescem e diminuem ao longo do comprimento.

As asas devem afunilar em direção às pontas. A parte superior vista de cima precisa parecer um avião, não uma plataforma.

6. Navios e barcos

Componentes: casco, proa, popa, convés, cabine quando aplicável, mastros, velas ou motores.

O casco deve afunilar na proa e normalmente na popa. A seção transversal muda ao longo do comprimento.

Para veleiros, o mastro deve nascer do convés e as velas devem ser planos verticais/diagonais, não paredes grossas.

Use perfis sucessivos para formar o casco; não use um cuboid único.

7. Cavernas naturais

Uma caverna é espaço negativo orgânico. Gere um caminho 3D, raio variável, ruído nas paredes, câmaras, estreitamentos, mudanças de altura e ramificações ocasionais.

A entrada deve se integrar ao terreno. Evite corte retangular na superfície.

O piso e teto não devem permanecer planos por longos trechos.

Use NOISE_CARVE e elipsoides sobrepostos ao longo de spline, mas varie direção, raio e altura.

8. Ilhas e montanhas

Uma ilha não é um prisma. Comece por footprint elíptico/irregular, crie camadas horizontais que mudam de raio por altura e aplique ruído nas bordas.

A parte inferior pode afunilar. A superfície deve ter relevo. Para ilha de pedra, use pedra dominante, mas permita variação natural.

Montanhas precisam de perfil vertical progressivo, cristas, encostas e ruído; evite pirâmide perfeita ou caixa.

9. Árvores

Árvore = tronco + ramificações + copa. O tronco não deve ser um poste totalmente reto em árvores grandes.

Crie ramificações principais em diferentes alturas e direções; a copa deve acompanhar os galhos, com massa irregular.

Tipos diferentes de árvore pedem silhuetas diferentes. Árvores de selva podem ter troncos altos, copa no topo, cipós e contrafortes.

10. Arenas

Arena é espaço jogável, não apenas recinto. Deve ter área útil, spawns, acesso, limites, circulação, acabamento e layout coerente com o modo.

Dimensione por jogadores. Para PvP, mantenha linhas de visão e cobertura proporcionais. Para mob arena, considere áreas de spawn e segurança.

Uma arena vazia com 4 paredes deve reprovar no validador se o usuário não pediu algo extremamente simples.

11. Bandeiras, logos e pixel art

Bandeiras e símbolos são composição 2D por padrão. Use painel de espessura 1, proporção correta e elementos gráficos principais.

Brasil: fundo verde, losango amarelo, círculo azul e faixa branca; URSS: fundo vermelho, foice e martelo/estrela em amarelo.

Cor dominante sozinha nunca basta. Se o resultado for apenas um retângulo colorido, reprovar.

12. StructurePlan recomendado

Todo objeto complexo deve virar um plano estruturado antes da execução.

```
{
"name": "aircraft_small",
"category": "AIRCRAFT",
"origin": [0,0,0],
"bounds": {"x": 42, "y": 14, "z": 36},
"required_components": [
"fuselage", "nose", "cockpit",
"left_wing", "right_wing",
"tail", "left_stabilizer", "right_stabilizer"
],
"operations": [
{"type":"LOFT_BETWEEN_PROFILES","component":"fuselage"},
{"type":"EXTRUDED_POLYGON","component":"left_wing"},
{"type":"EXTRUDED_POLYGON","component":"right_wing"}
],
"estimated_blocks": 4600
}
```

O plano deve guardar a relação entre operações e componentes. Isso permite verificar se o avião possui asas, se a casa possui telhado, se a arena possui entrada etc.

13. Validação antes da execução

Métrica	Pergunta
component_coverage	Todos os componentes essenciais existem no plano?
boxiness_score	A estrutura é dominada por 1-2 prismas grandes sem justificativa?
silhouette_score	A silhueta é reconhecível para a categoria?
proportion_score	As proporções básicas fazem sentido?
shape_diversity	Há variedade geométrica suficiente?
playability	Se for arena/evento, dá para jogar?
terrain_fit	A origem e o terreno são apropriados?

Regra: se component_coverage falhar, ou boxiness for alto para uma categoria não retangular, não construir. Gere um plano novo.

14. Boxiness: proteção contra 'tudo vira caixa'

Um plano é suspeito quando a maior parte do volume visual vem de um ou dois cuboids grandes e isso elimina a forma do objeto.

```
EXEMPLOS QUE DEVEM FALHAR:
AIRCRAFT = 1 cuboid comprido + 2 pisos retangulares
ISLAND = 1 cuboid 50x50x30
FLAG = hollow_cuboid verde/vermelho
ARENA = 4 paredes + piso, sem acesso/spawns/design
CAVE = caixa de AIR

EXEMPLOS ACEITÁVEIS:
HOUSE = vários volumes retangulares + telhado + aberturas + interior
APARTMENT = módulos repetidos, corredores, escadas e fachada
WAREHOUSE = volume retangular, pois a categoria justifica
```

15. Execução segura

• Faça operações pesadas em lotes por tick. Nunca altere centenas de milhares de blocos no mesmo tick.

• Antes de cada bloco alterado, registre o BlockData anterior no BuildJournal/BuildRegistry.

• Após a execução, registre estado final, bounds e componentes concluídos.

• Se uma operação falhar no meio, pare dependências posteriores e faça rollback quando apropriado.

• Não diga 'pronto' enquanto a verificação final não passar.

16. Undo e destruição correta

Se a IA criou uma construção, destruir/remover deve significar restaurar o estado anterior exato, não preencher tudo com AIR.

```
create_build -> save original block states -> execute -> verify
"destrua a casa" -> resolve recent HOUSE -> undo_build(build_id)
-> restore original states -> verify -> mark UNDONE
```

Se houve alterações manuais depois, detectar conflito antes de sobrescrever.

17. Aprendizado com schematics

Schematics devem ser usados como referências estruturais. Extraia bounds, paleta, densidade, simetria, perfis, componentes, relações espaciais e estilo.

Não memorize apenas coordenadas. Transforme a construção em características generalizáveis.

```
SCHEMATIC DE AVIÃO -> CONHECIMENTO
- comprimento relativo
- perfil da fuselagem por seção
- posição/envergadura das asas
- forma da cauda
- distribuição de materiais
- relação cockpit/fuselagem
- simetria lateral
- motores e trem de pouso
```

18. Aprendizado com feedback

Feedback do administrador deve virar lição generalizável.

Feedback	Lição correta
'o avião ficou quadrado'	AIRCRAFT: reduzir boxiness, usar fuselagem afunilada, asas e cauda reconhecíveis
'a bandeira virou uma caixa'	FLAG: usar composição 2D, espessura 1, preservar símbolos
'a caverna parece um buraco'	CAVE: usar spline 3D, câmaras, variação de raio e integração com terreno
'a arena ficou vazia'	ARENA: exigir spawns, acesso, layout e elementos jogáveis

19. Auto-revisão

Antes de executar uma build complexa, faça uma revisão do plano. Perguntas obrigatórias:

• Se eu visse apenas a silhueta, reconheceria o objeto?

• Há componentes essenciais ausentes?

• As proporções estão coerentes?

• Estou usando cuboids como atalho para algo que deveria ter forma própria?

• O objeto está colocado de forma segura e coerente com o terreno?

• Se o usuário pediu funcionalidade, ela existe de verdade?

20. Exemplos de decomposição

Pedido	Decomposição mínima
'faça um avião comercial'	fuselagem, nariz, cockpit, asas, motores, cauda, estabilizadores, trem de pouso
'faça um barco de vela'	casco, convés, proa, popa, mastro, velas, cabine/detalhes
'faça uma igreja medieval'	nave, entrada, torre, telhado, janelas, interior, iluminação
'faça uma ponte suspensa'	endpoints, pilares, tabuleiro, cabos, guarda-corpo, acesso
'faça uma arena para 100'	capacidade, dimensões, múltiplos spawns, acesso, obstáculos, limites, saída
'faça uma caverna natural'	entrada, túnel curvo, câmaras, variação, ramificações, integração

21. Critério de sucesso

Uma construção só é sucesso quando **o resultado corresponde semanticamente ao pedido**. Quantidade de blocos e ausência de exceções não bastam.

```
SUCESSO = execução técnica
+ componentes essenciais
+ proporções
+ silhueta
+ funcionalidade
+ verificação pós-build
```

22. Regras finais para o agente

• Não invente que uma estrutura está pronta se apenas uma parte foi criada.

• Não substitua um objeto complexo por uma caixa porque é mais fácil.

• Quando não souber construir, procure referências na KnowledgeBase/schematics antes de improvisar.

• Se uma referência ensinada pelo administrador existir, use-a como contexto prioritário.

• Se um plano falhar na validação, revise. Não peça confirmação do usuário para correções óbvias.

• Quando o usuário disser que algo ficou errado, associe o feedback à build anterior e gere uma lição persistente.

• Prefira qualidade e coerência a velocidade em tarefas complexas.

• Comandos simples devem continuar rápidos; construções complexas podem usar várias etapas de planejamento e revisão.

Apêndice A - Checklist rápido por categoria

Categoria	Checklist mínimo
HOUSE	entrada, janelas, telhado, interior, piso, circulação
AIRCRAFT	fuselagem, nariz, asas L/R, cauda, estabilizadores, proporção
SHIP	casco afunilado, proa, popa, convés, propulsão/vela
CAVE	entrada, continuidade, raio variável, curvas, câmara, ruído
ISLAND	footprint irregular, camadas variáveis, bordas, relevo, integração
TREE	tronco, galhos, copa, assimetria
ARENA	área útil, spawns, acesso, limites, layout, escala
FLAG	painel 2D, proporção, cores, símbolos principais

Apêndice B - Frase de política recomendada para o system prompt

```
Para construções: pense primeiro no objeto real, depois converta para geometria Minecraft.
As tools são primitivas de construção, não o limite do que pode ser criado.
Se não existir uma ferramenta específica, decomponha o objeto em componentes e formas.
Nunca use uma caixa genérica como substituto para uma forma semântica própria.
Valide componentes, proporções e silhueta antes de executar.
Depois de construir, verifique o resultado e corrija divergências antes de afirmar sucesso.
```

Fim do manual.